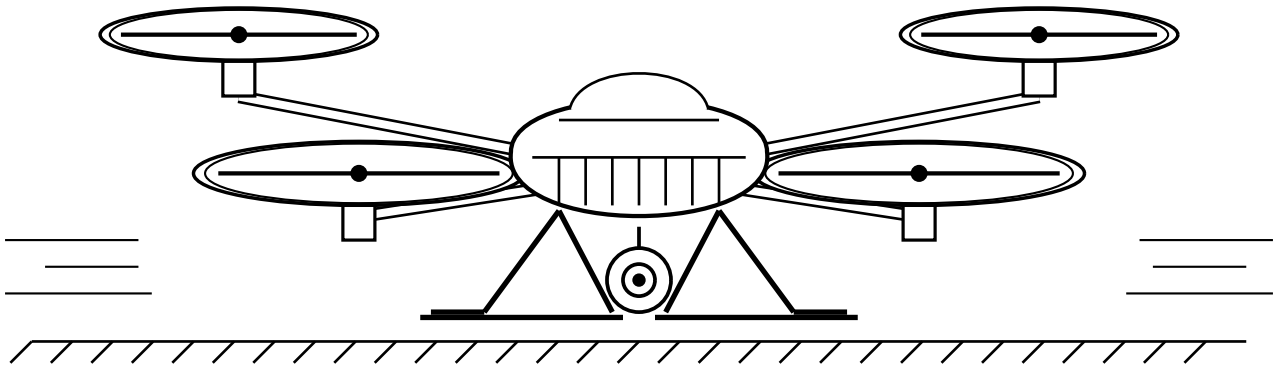
A PROJECT REPORT
ON

DRONE FLEET MANAGEMENT AND MISSION SIMULATION SYSTEM



A C++ based console application for simulated drone fleet management and mission assignment using Object-Oriented Programming concepts

Name	**:**	Soumyakanta Sahu
ID	**:**	B125125
Branch	**:**	CSEB-2
Semester	**:**	3rd Semester
Subject	**:**	Object-Oriented Programming (OOP)
Project Title	**:**	Drone Fleet Management and Mission Simulation System
Instructor	**:**	Dr. Sobhanayak Srichandan

FINAL PROJECT REPORT

Drone Fleet Management and Mission Simulation System

Name: Soumyakanta Sahu | ID: B125125 | Branch: CSEB-2 | Subject: Object-Oriented Programming (3rd Semester) | Instructor: Dr. Sobhanayak Srichandan

Contents

Abstract . . . 2
1. Introduction . . . 2
2. Problem Statement . . . 3
3. Objectives . . . 3
4. Scope and Limitations . . . 3
5. Tools and Environment . . . 3
6. Requirements . . . 3
7. Methodology . . . 4
8. System Design . . . 4
9. Status Lifecycles . . . 5
10. Workflow . . . 6
11. Algorithms . . . 6
12. Battery Model (Simulation) . . . 6
13. File Format and Persistence . . . 7
14. Concepts Used . . . 7
15. Implementation Explanation . . . 7
16. Source Code (drone_fleet_final.cpp) . . . 8
17. Output . . . 17
18. Testing and Validation . . . 19
19. Limitations and Future Scope . . . 20
20. Challenges Faced and Learning Outcomes . . . 20
21. Conclusion . . . 20

Abstract

This report presents the **Drone Fleet Management and Mission Simulation System**, a menu-driven, console-based application written in C++17 for the Object-Oriented Programming (OOP) course. The program manages a simulated fleet of drones and the missions assigned to them. A user can register, search and remove drones; create missions with a priority; assign a suitable drone after automatic checks of availability, payload, range and battery; complete missions using a simplified battery-consumption model; charge drones; record maintenance; view fleet statistics; and save or reload every record through text files. Delivery, Surveillance, Mapping and Rescue drones are modelled as classes derived from an abstract *Drone* class, and a *FleetManager* class coordinates all operations. The report explains the problem, objectives, design, algorithms, OOP concepts, complete source code, program output and testing. The project is an academic simulation: it does not control real drones, use hardware or use live GPS data.

1. Introduction

1.1 Background

Organisations that operate several drones must know, at any moment, what each drone can carry, how far it can fly, how much charge it has, whether it is free, charging or being serviced, and which task it is performing. Keeping this information in notebooks or separate spreadsheets is slow and error-prone. A small software system that stores the data in one place and applies the same rules every time removes much of that risk.

1.2 Why Object-Oriented Programming

OOP organises a program around the real-world things it describes. Each thing becomes a class that holds its own data and the operations that are allowed on it. In a drone fleet the natural objects are drones, missions, maintenance records and the fleet manager. Inheritance lets the four drone types share common behaviour, while polymorphism lets each type respond in its own way. Encapsulation keeps the data safe from accidental change. These ideas make the program easier to understand, test and extend.

1.3 About this project

The project is a console application that runs in the VS Code terminal and uses only the C++ standard library. All information is simulated. The program was built in small stages and tested after each one, and it finally supports twelve menu operations plus Exit, with automatic loading and saving of records.

2. Problem Statement

Managing several drones requires tracking their specifications, battery levels, availability, missions and maintenance. When these details are kept separately, it is difficult to decide quickly which drone is suitable for a particular task. The problem addressed here is to organise these operations into a single C++ program, applying OOP principles to structure the system, so that:

- every drone, mission and maintenance record is stored consistently and cannot be created with invalid data;
- a suitable drone is selected by clear, repeatable rules instead of guesswork, and the reason is shown when none is suitable;
- statuses (AVAILABLE, ON_MISSION, MAINTENANCE, PENDING, ASSIGNED, COMPLETED) change automatically and consistently;
- the information survives between runs and the program stays running even when the user makes mistakes.

3. Objectives

1. **Develop a menu-driven C++ application for managing a simulated drone fleet.** A numbered menu with twelve operations and Exit, repeated until the user quits.
2. **Register, view, search and manage different types of drones.** Four drone types created through one common interface; search by ID; removal with confirmation.
3. **Create missions and assign suitable drones using battery, range, payload and availability checks.** Priority-based selection with four checks and a clear reason for every rejected drone.
4. **Simulate mission completion, battery consumption, charging and maintenance.** A simplified battery model, charging up to 100% and maintenance records with date and description.
5. **Demonstrate OOP concepts, STL containers, exception handling and file handling.** Classes, inheritance, virtual functions, map, vector, priority_queue, try/catch and text-file storage.

4. Scope and Limitations

In scope (version 1)	Out of scope
Terminal-based C++ application; menu-driven interaction	Physical drone control or connection to drone hardware
Simulated data and simplified operational rules	Live GPS tracking and real-time flight systems
Drone, mission and maintenance management; statistics	External APIs, web interface, mobile app, graphical interface
Saving and loading records in text files	Database integration and online services

Battery use is computed with a simple formula and must not be read as a real flight-performance prediction. The limited scope keeps the project suitable for an academic OOP implementation and allows the focus to stay on class design, program logic and data management.

5. Tools and Environment

Item	Details
Language / standard	C++17 (std::make_unique, static constexpr members, structured result types)
Compiler and flags	g++ with -std=c++17 -Wall -Wextra -pedantic (compiles without warnings)
Editor / environment	Visual Studio Code integrated terminal; console input and output only
Standard headers	iostream, iomanip, vector, map, queue, memory, utility, stdexcept, sstream, fstream, ctime, cmath, cctype, algorithm, cstdlib
Persistent storage	Three plain text files in the working folder: drones.txt, missions.txt, maintenance.txt
Testing aids	Scripted input sessions; AddressSanitizer and UndefinedBehaviorSanitizer builds

6. Requirements

ID	Functional requirement	Menu
FR1	Register a drone with ID, model, type, battery, range, payload capacity and speed; reject duplicates and invalid values	1
FR2	Display all drones in a table with type and status	2
FR3	Search a drone by ID	3
FR4	Remove a drone after confirmation; refuse while it is ON_MISSION	4
FR5	Create a mission with type, destination, distance, payload and priority; initial status PENDING	5
FR6	Display all missions with status and assigned drone	6

ID	Functional requirement	Menu
FR7	Assign a mission to a suitable drone (availability, payload, range, battery) or keep it PENDING with reasons	7
FR8	Complete an assigned mission: update battery, mission status and drone status	8
FR9	Charge a drone (not during a mission or maintenance; maximum 100%)	9
FR10	Record maintenance, keep a history, and mark it finished; a drone in maintenance is not assignable	10
FR11	Show fleet statistics (counts by type and status, average battery, missions, distance, most active drone)	11
FR12	Save records to files; load them at start-up; save again on exit	12 / 0

Non-functional requirements

- **Reliability:** invalid input or illegal operations never crash the program; they produce a clear message.
- **Usability:** numbered menu, labelled prompts, aligned tables and messages that explain what happened and why.
- **Maintainability:** separate classes with single responsibilities; rules enforced in one place (FleetManager).
- **Portability:** standard C++17 only, so it builds with g++ on Windows, Linux and macOS.
- **Data safety:** saved files are validated when loaded; bad lines are skipped and counted.

7. Methodology

The system was developed with an incremental, stage-by-stage method. Requirements were first listed from the problem statement; the real-world objects were then identified and turned into classes; and the program was built in eight stages. After every stage it was compiled with warnings enabled and tested with valid and invalid input, so that a fault could be traced to the most recent change. Rules that must always hold were placed in constructors and in FleetManager so that no menu function can break them.

Stage	Work carried out	Main concept practised
1	Menu skeleton with a validated input loop	Control flow, input validation
2	Drone class: private data, constructor with validation, getters	Encapsulation, constructors, enum class
3	Abstract base class and four derived drone classes	Inheritance, abstraction, polymorphism
4	FleetManager: register, view, search and remove drones	STL map, smart pointers, exceptions
5	Mission class: create and view missions	Classes, validation, priority and status enums
6	Suitability checks and mission assignment	Virtual functions, priority_queue
7	Completion, charging and maintenance	State changes, MaintenanceRecord
8	Fleet statistics, saving and loading records	File streams, parsing, validation

8. System Design

The program has three layers. The **menu and input layer** (free functions) talks to the user and validates what is typed. The **logic layer** (the *FleetManager* class) owns all data and enforces all rules. The **data layer** consists of the domain classes (Drone and its subclasses, Mission, MaintenanceRecord) and the three text files. Fig. 1 shows the classes and their relations.

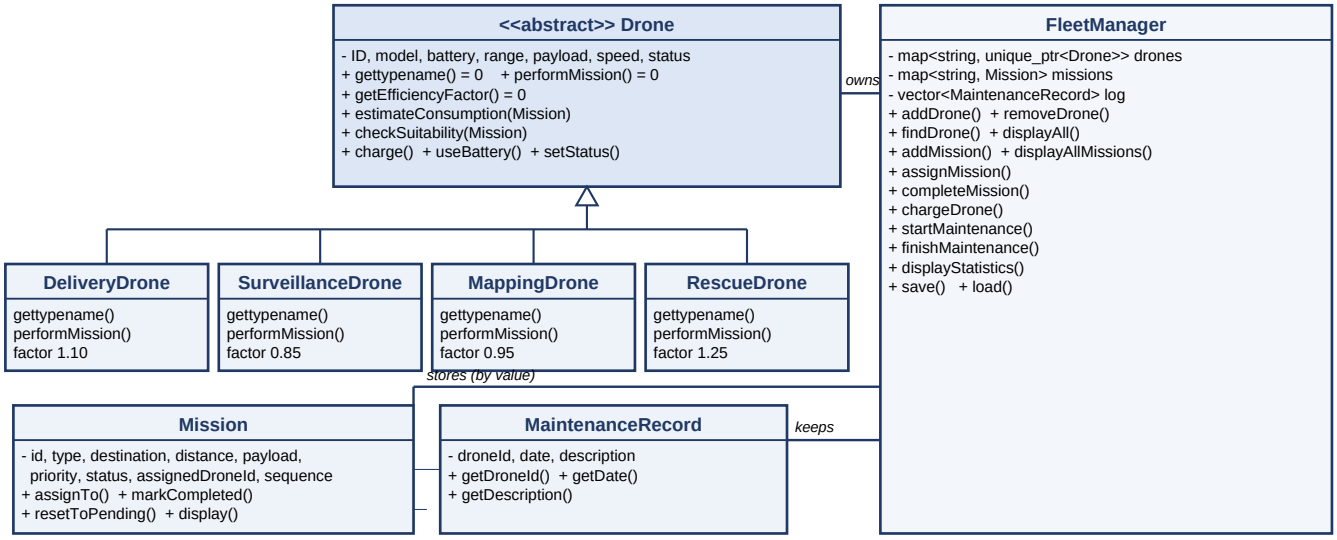


Fig. 1: Class diagram of the system

Class	Data members	Main member functions	Responsibility
Drone (abstract)	ID, model, battery, range, payload, speed, status	gettypename(), performMission(), getEfficiencyFactor() [pure virtual]; estimateConsumption(), checkSuitability(), charge(), useBattery(), setStatus(), display(), getters	Common data and behaviour of all drones; validates data in its constructor

Class	Data members	Main member functions	Responsibility
DeliveryDrone, SurveillanceDrone, MappingDrone, RescueDrone	(inherited)	Override the three virtual functions; battery factors 1.10, 0.85, 0.95 and 1.25	Type-specific behaviour and messages
Mission	id, type, destination, distance, payload, priority, status, assignedDroneId, sequence	assignTo(), markCompleted(), resetToPending(), display(), getters	One task and its lifecycle
MaintenanceRecord	droneId, date, description	Two constructors (new record / loaded record), getters	One entry of a drone's service history
FleetManager	drones (map of unique_ptr), missions (map), nextSequence, maintenanceLog (vector)	addDrone, removeDrone, findDrone, addMission, assignMission, completeMission, chargeDrone, startMaintenance, finishMaintenance, displayStatistics, save, load; private: requireDrone, loadDroneLine, loadMissionLine, reconcile	Coordinates every operation and enforces all rules
Helper types	AssignmentResult, CompletionResult, LoadSummary, InputClosed	-	Carry results back to the menu layer

9. Status Lifecycles

Drones and missions each follow a small set of statuses. A drone is **AVAILABLE** when it can take work, **ON_MISSION** while flying a mission, and **MAINTENANCE** while being serviced. A mission is **PENDING** until a drone is assigned, **ASSIGNED** while its drone is flying, and **COMPLETED** afterwards. Status changes always happen in pairs inside FleetManager (for example ASSIGNED together with ON_MISSION) so the two records never disagree.

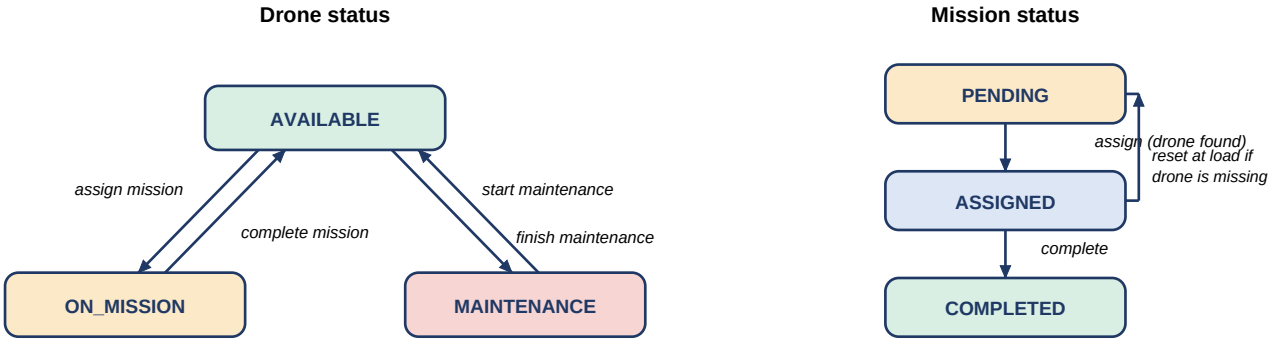


Fig. 2: Status lifecycles of drones and missions

10. Workflow

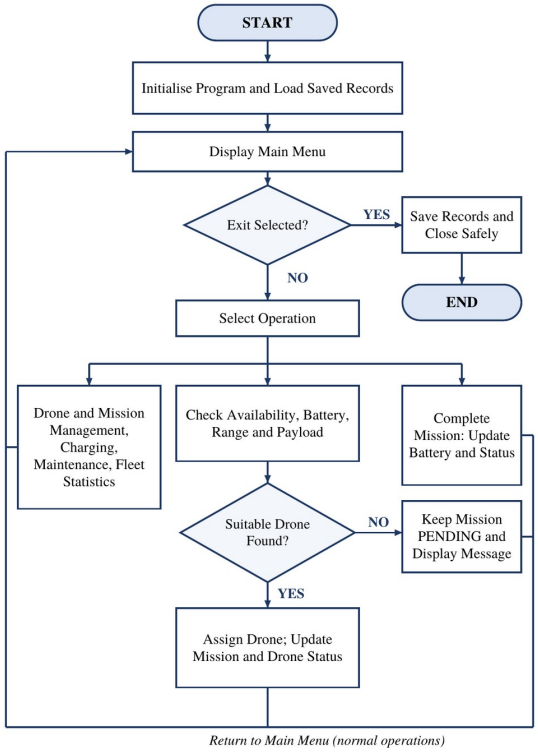


Fig. 3: Simplified overall workflow

The program loads saved records at start-up and then repeatedly shows the menu. Each choice calls a menu function inside a try/catch block, so an error produces a message and the menu returns. Normal operations return to the main menu; the Exit option saves all records and ends the program. The same save also runs if the input stream is closed.

Assigning a mission is the central operation. The mission is chosen by the user, or automatically as the highest-priority PENDING mission. Every drone is then checked in a fixed order (status, payload, range, battery). A drone that fails is skipped and the reason is remembered. If no drone passes, the mission stays PENDING and all reasons are displayed. If several pass, the one whose type matches the mission is preferred, and then the one with more battery.

Completing a mission estimates the battery used, calls the drone's own performMission(), lowers the battery (never below 0%), sets the mission COMPLETED and the drone AVAILABLE. A warning is printed if the remaining battery is below 30%.

Charging and maintenance are refused when the drone is on a mission, and a drone in maintenance cannot be charged or assigned.

11. Algorithms

```
ASSIGN-MISSION(missionId)
    mission <- find missionId                          // error if it does not exist
    if mission.status is not PENDING: error "already <status>"
    best <- none
    for each drone d in the fleet (in ID order):
        reason <- CHECK-SUITABILITY(d, mission)
        if reason is not empty: remember "d.ID: reason"; continue
        if best is none: best <- d; continue
        if d matches the mission type and best does not: best <- d
        else if both or neither match and d.battery > best.battery: best <- d
    if best is none: return "no suitable drone" with the remembered reasons   // mission stays PENDING
    best.status <- ON_MISSION;  mission.status <- ASSIGNED;  mission.droneId <- best.ID
    return best.ID

CHECK-SUITABILITY(drone, mission)
    if drone.status is not AVAILABLE:          return "not available (STATUS)"
    if mission.payload  > drone.payloadCapacity: return "payload too high"
    if mission.distance > drone.range:           return "range too short"
    needed <- CONSUMPTION(drone, mission) + 10                  // 10% safety reserve
    if drone.battery < needed:                 return "battery too low"
    return ""                                                   // suitable

COMPLETE-MISSION(missionId)
    mission <- find missionId;  error unless mission.status is ASSIGNED
    drone <- find mission.droneId;   used <- CONSUMPTION(drone, mission)
    drone.performMission(mission.destination)                   // virtual call: type-specific message
    drone.battery <- max(0, drone.battery - used)
    mission.status <- COMPLETED;  drone.status <- AVAILABLE
```

12. Battery Model (Simulation)

The battery used by a mission is estimated with a simple teaching model: **consumption (%) = (distance / range) × 100 × type factor × (1 + 0.5 × mission payload / drone payload capacity)**. The type factor is 1.10 for Delivery, 0.85 for Surveillance, 0.95 for Mapping and 1.25 for Rescue drones (higher means more battery use). A drone is suitable only if its battery covers the consumption plus a 10% reserve. The model is not a real flight-performance calculation.

Case	Drone and mission	Calculation	Result
A	D1 Delivery, battery 90%, range 20 km, capacity 5 kg; mission M1: 8 km, 2 kg	8/20×100 = 40; ×1.10 = 44; load 1 + 0.5×2/5 = 1.2; 44×1.2 = 52.8%; needs 62.8%	Accepted; 37.2% left
B	S1 Surveillance, battery 80%, range 15 km, capacity 1 kg; mission: 12 km, 0.5 kg	12/15×100 = 80; ×0.85 = 68; load 1.25; 68×1.25 = 85.0%; needs 95.0%	Rejected: battery too low
C	R1 Rescue, battery 100%, range 30 km, capacity 10 kg; mission M2: 12 km, 4 kg	12/30×100 = 40; ×1.25 = 50; load 1.2; 50×1.2 = 60%; needs 70%	Accepted; 40% left
D	Any drone; mission M3: 40 km, 0 kg (ranges 20, 15 and 30 km)	distance 40 km is above every drone's range	Rejected: range too short

13. File Format and Persistence

Every record is one text line with fields separated by '|'. A typed '|' is replaced by '/' when text is read, so the format cannot be broken. Enumerations are stored as numbers. At start-up each line is checked inside its own try/catch: wrong field counts, unknown types, invalid numbers, duplicate IDs and out-of-range values are skipped and counted. Loaded objects pass through the normal constructors, so all validation rules also apply to saved data. Finally *reconcile()* repairs links, for example a mission assigned to a drone that no longer exists returns to PENDING.

File	Fields (in order)	Example line
drones.txt	type, ID, model, battery, range, payload, speed, status	Delivery|D1|Falcon X|77.2|20|5|60|0
missions.txt	ID, type, destination, distance, payload, priority, status, drone ID, sequence	M1|1|Sector 7|8|2|3|2|D1|1
maintenance.txt	drone ID, date and time, description	S1|2026-10-28 00:59|Camera check

14. Concepts Used

Concept	Where it is used	Explanation
Encapsulation	Drone, Mission, MaintenanceRecord	Data members are private; const getters read them; only meaningful functions (charge, useBattery, assignTo, markCompleted) change them.
Inheritance	Delivery/Surveillance/Mapping/Rescue Drone : Drone	Derived classes reuse the data and functions of Drone and add their own behaviour.
Abstraction	Drone	Pure virtual functions make Drone abstract: only concrete drone types can be created.
Runtime polymorphism	completeMission(), displayStatistics()	Virtual calls through Drone pointers run the version of the real drone type.
Constructors	all classes	Initialise objects and validate data, so an invalid drone or mission cannot exist.
Virtual destructor	Drone	Ensures derived objects are destroyed correctly when deleted through a base pointer.
Exception handling	constructors, FleetManager, main()	invalid_argument and runtime_error are thrown and caught once per menu action; InputClosed handles end of input.
STL map	drones, missions	Key = ID gives fast lookup, duplicate rejection and sorted display.
STL vector	maintenanceLog, reasons, parsed fields	Dynamic list of records, rejection reasons and split text fields.
STL priority_queue	highestPriorityPendingId()	Keeps the most urgent PENDING mission on top (priority first, then older mission).
Smart pointers	map of unique_ptr<Drone>	Automatic ownership and cleanup (RAII); no manual new or delete.
File handling	save(), load()	ofstream writes and ifstream reads the record files line by line.
String streams	split(), fmt()	istringstream splits lines; ostringstream formats numbers.
Other features	enum class, static constexpr, const, references, structs	Type-safe statuses, a shared reserve constant, const-correct functions and result structs.

15. Implementation Explanation

Program flow. *main()* creates one FleetManager, loads saved records and runs a loop: show the menu, read a validated choice (0-12), call the matching function inside try/catch, and repeat until 0. On the way out, and also if the input stream closes, the records are saved.

Input layer. All input is read with getline and converted by helpers (readText, readNumber, readInt, readOptionalText, readYesNo) that re-ask until the answer is valid, refuse empty text and non-finite numbers such as *nan*, and avoid the leftover-newline problem of mixing cin >> with getline. IDs are converted to upper case, so d1 and D1 are the same drone.

Drone and Mission. A Drone validates its data and starts AVAILABLE. checkSuitability() returns an empty string when the drone fits a mission, otherwise the reason. A Mission starts PENDING with no drone; its status changes only through assignTo(), markCompleted() and resetToPending().

FleetManager rules. Duplicate IDs are rejected; a drone that is ON_MISSION cannot be removed, charged or sent for maintenance; a drone in maintenance cannot be assigned or charged; charging never exceeds 100%; only ASSIGNED missions can be completed; only PENDING missions can be assigned.

Statistics. The program counts drones by type and status, averages the battery, counts missions by status, adds the distance of completed missions, finds the drone with the most completed missions and reports the number of maintenance records.

Persistence. save() writes the three files and throws a runtime_error if a file cannot be written; load() reads them, validating every line. A missing file simply means a first run.

16. Source Code (drone_fleet_final.cpp)

Complete program, 1010 lines. Line numbers refer to the submitted source file (blank lines are omitted to save space).

```
#include<iostream>
#include<vector>
#include<stdexcept>
#include<iomanip>
#include<map>
#include<memory>
#include<cctype>      // toupper
#include<cstdlib>     // exit
#include<utility>     // move
#include<sstream>     // ostringstream, istringstream
#include<queue>       // priority_queue
#include<ctime>       // time, strftime
#include<fstream>     // ifstream, ofstream
#include<algorithm>   // replace
#include<cmath>       // isfinite
using namespace std;
enum class DroneStatus{AVAILABLE,ON_MISSION,MAINTENANCE};
enum class MissionStatus{PENDING,ASSIGNED,COMPLETED};
enum class Priority{LOW=1,MEDIUM,HIGH};
enum class MissionType{DELIVERY=1,SURVEILLANCE,MAPPING,RESCUE};
string missionStatusToString(MissionStatus s){
    switch(s){
        case MissionStatus::PENDING: return "PENDING";
        case MissionStatus::ASSIGNED: return "ASSIGNED";
        case MissionStatus::COMPLETED: return "COMPLETED";
    }
    return "UNKNOWN";
}
string priorityToString(Priority p){
    switch(p){
        case Priority::LOW: return "LOW";
        case Priority::MEDIUM: return "MEDIUM";
        case Priority::HIGH: return "HIGH";
    }
    return "UNKNOWN";
}
string missionTypeToString(MissionType t){
    switch(t){
        case MissionType::DELIVERY:  return "Delivery";
        case MissionType::SURVEILLANCE: return "Surveillance";
        case MissionType::MAPPING:  return "Mapping";
        case MissionType::RESCUE:  return "Rescue";
    }
    return "Unknown";
}
void printMissionHeader(){
    cout<<left<<setw(8)<<"ID"<<setw(14)<<"Type"<<setw(16)<<"Destination"
        <<setw(8)<<"Dist"<<setw(9)<<"Payload"<<setw(10)<<"Priority"
        <<setw(11)<<"Status"<<"Drone"<<endl;
    cout<<string(86,'-')<<endl;
}
class Mission{
    string id;
    MissionType type;
    string destination;
    double distance;
    double payload;
    Priority priority;
    MissionStatus status;
    string assignedDroneId; //empty while PENDING
    int sequence;           //creation order
    public:
        Mission(string id,MissionType type,string destination,double distance,double payload,Priority priority,int sequence)
                :id(id),type(type),destination(destination),distance(distance),payload(payload),
                 priority(priority),status(MissionStatus::PENDING),sequence(sequence){
            if(id.empty())
                throw invalid_argument("Mission ID cannot be empty");
            if(distance<=0)
                throw invalid_argument("Distance must be greater than zero");
            if(payload<0)
                throw invalid_argument("Payload cannot be negative");
        }
        string getId() const {return id;}
        MissionType getType() const { return type;}
        string getDestination() const {return destination;}
        double getDistance() const{ return distance;}
        double getPayload() const{ return payload;}
        Priority getPriority() const { return priority; }
        MissionStatus getStatus() const{return status;}
        string getDroneId() const { return assignedDroneId; }
        int getSequence() const { return sequence; }
        void assignTo(const string& droneId){
            status=MissionStatus::ASSIGNED;
            assignedDroneId=droneId;
        }
        void markCompleted(){status=MissionStatus::COMPLETED; }
        void resetToPending(){
            status=MissionStatus::PENDING;
            assignedDroneId.clear();
        }
        void display() const{
            cout<<left<<setw(8)<<id
                <<setw(14)<<missionTypeToString(type)
                <<setw(16)<<destination
                <<setw(8)<<distance
                <<setw(9)<<payload
                <<setw(10)<<priorityToString(priority)
                <<setw(11)<<missionStatusToString(status)
                <<(assignedDroneId.empty()?"-":assignedDroneId)<<endl;
        }
```

```
};
//printDroneHeader() function
void printDroneHeader(){
    cout<<left<<setw(8)<<"ID"<<setw(14)<<"Model"<<setw(14)<<"Type"
            <<setw(8)<<"Batt%"<<setw(8)<<"Range"<<setw(8)<<"Payload"
            <<setw(8)<<"Speed"<<"Status"<<endl;
    cout<<string(86,'-')<<endl;
}
struct InputClosed {};      // thrown when the input stream ends (Ctrl+D / Ctrl+Z)
//Helper functions
string toUpper(string s){
    for(char& c : s) c= toupper(static_cast<unsigned char>(c));
    return s;
}
string readText(const string& prompt){
    string s;
    while(true){
        cout<<prompt;
        if(!getline(cin,s)) throw InputClosed();
        replace(s.begin(), s.end(), '|', '/');   // '|' separates fields in the save files
        if(!s.empty()) return s;
        cout<<"Input cannot be empty.\n";
    }
}
double readNumber(const string& prompt){
    string s;
    while(true){
        cout<<prompt;
        if(!getline(cin,s)) throw InputClosed();
        try{
            size_t pos;
            double v=stod(s,&pos);
            if(pos==s.size() && isfinite(v)) return v; //whole text was a real number
        } catch(const exception&){}
        cout<<"Please enter a valid number.\n";
    }
}
int readInt(const string& prompt,int lo,int hi){
    string s;
    while(true){
        cout<<prompt;
        if(!getline(cin,s)) throw InputClosed();
        try{
            size_t pos;
            int v=stoi(s,&pos);
            if(pos==s.size() && v>=lo && v<=hi) return v;
        }catch(const exception&){}
        cout<<"Please enter a whole number from "<<lo<<" to "<<hi<<".\n";
    }
}
string readOptionalText(const string& prompt) {      // empty answer is allowed
    string s;
    cout << prompt;
    if (!getline(cin, s)) throw InputClosed();
    return s;
}
//formats a number with 1 decimal place
string fmt(double v){
    ostringstream out;
    out<<fixed<<setprecision(1)<<v;
    return out.str();
}
string statustoString(DroneStatus s){
    switch(s){
        case DroneStatus::AVAILABLE: return "AVAILABLE";
        case DroneStatus::ON_MISSION: return "ON_MISSION";
        case DroneStatus::MAINTENANCE: return "MAINTENANCE";
    }
    return "UNKNOWN";
}
bool readYesNo(const string& prompt) {
    while (true) {
        string a = toUpper(readText(prompt + " (y/n): "));
        if (a == "Y" || a == "YES") return true;
        if (a == "N" || a == "NO") return false;
        cout << "Please answer y or n.\n";
    }
}
string currentTime() {
    time_t t = time(nullptr);
    char buffer[32];
    strftime(buffer, sizeof buffer, "%Y-%m-%d %H:%M", localtime(&t));
    return buffer;
}
// ----- helpers for reading the save files -----
vector<string> split(const string& line, char delimiter) {
    vector<string> parts;
    string field;
    istringstream stream(line);
    while (getline(stream, field, delimiter)) parts.push_back(field);
    if (!line.empty() && line.back() == delimiter) parts.push_back("");
    return parts;
}
double parseNumber(const string& s) {
    size_t pos;
    double v = stod(s, &pos);
    if (pos != s.size() || !isfinite(v)) throw invalid_argument("bad number");
    return v;
}
int parseInt(const string& s) {
    size_t pos;
    int v = stoi(s, &pos);
```

```
    if (pos != s.size()) throw invalid_argument("bad number");
    return v;
}
class Drone{
    string ID;
    string model;
    double battery;
    double range;
    double payload;
    double speed;
    DroneStatus status;
    public:
        Drone(string id,string model,double battery,double range,double payload,double speed): ID(id),model(model),battery(battery),
        range(range),payload(payload),speed(speed),status(DroneStatus::AVAILABLE){
            if (battery < 0 || battery > 100)
                throw invalid_argument("Battery must be 0-100");
            if (range <= 0 || speed <= 0)
                throw invalid_argument("Range and speed must be positive");
            if (payload < 0)
                throw invalid_argument("Payload cannot be negative");
        }
        virtual ~Drone(){}   //virtual destructor
        virtual string gettypename() const=0; //pure virtual function
        virtual void performMission(const string&destination)=0; //pure virtual function
        virtual double getEfficiencyFactor() const=0; //pure virtual (battery use)
        string getId()const {return ID;}
        string getModel() const {return model;}
        double getBattery() const{return battery;}
        double Range() const {return range;}
        double Payload() const{return payload;}
        double Speed() const{return speed;}
        DroneStatus getStatus() const{return status;}
        void setStatus(DroneStatus s) { status = s; }
        void charge(double amount) {
            battery += amount;
            if (battery > 100) battery = 100;   // cap at 100
        }
        void useBattery(double amount) {
            battery -= amount;
            if (battery < 0) battery = 0;       // never negative
        }
        static constexpr double RESERVE_BATTERY = 10.0;   // % that must remain after a mission
        // Simplified battery model: share of range used, scaled by drone type and load
        double estimateConsumption(const Mission& m) const {
            double base = (m.getDistance() / range) * 100.0 * getEfficiencyFactor();
            double load = (payload > 0) ? 1.0 + 0.5 * (m.getPayload() / payload) : 1.0;
            return base * load;
        }
        // Returns "" if the drone can take the mission, otherwise the reason it cannot
        string checkSuitability(const Mission& m) const {
            if (status != DroneStatus::AVAILABLE)
                return "not available (" + statustoString(status) + ")";
            if (m.getPayload() > payload)
                return "payload too high (max " + fmt(payload) + " kg)";
            if (m.getDistance() > range)
                return "range too short (max " + fmt(range) + " km)";
            double needed = estimateConsumption(m) + RESERVE_BATTERY;
            if (battery < needed)
                return "battery too low (needs " + fmt(needed) + "%, has " + fmt(battery) + "%)";
            return "";
        }
        void display() const{
            cout<<left<<setw(8)<<ID
            <<setw(14)<<model
            <<setw(14)<<gettypename()
            <<setw(8)<<battery
            <<setw(8)<<range
            <<setw(8)<<payload
            <<setw(8)<<speed
            <<statustoString(status)<<endl;
        }
};
class DeliveryDrone: public Drone{
public:
    DeliveryDrone(string id,string model,double battery,double range,
                      double payload,double speed):Drone(id,model,battery,range,payload,speed){}
    string gettypename() const override{ return "Delivery";}
    void performMission(const string& destination) override{
        cout<<getId()<<"(Delivery): delivering package to "<<destination<<endl;
    }
    double getEfficiencyFactor() const override { return 1.10; }
};
class SurveillanceDrone: public Drone{
public:
    SurveillanceDrone(string id,string model,double battery,double range,
                        double payload,double speed):Drone(id,model,battery,range,payload,speed){}
    string gettypename() const override{ return "Surveillance";}
    void performMission(const string& destination) override{
        cout<<getId()<<"(Surveillance): patrolling "<<destination<<endl;
    }
    double getEfficiencyFactor() const override { return 0.85; }
};
class MappingDrone: public Drone{
public:
    MappingDrone(string id,string model,double battery,double range,
                        double payload,double speed):Drone(id,model,battery,range,payload,speed){}
    string gettypename() const override{ return "Mapping";}
    void performMission(const string& destination) override{
        cout<<getId()<<"(Mapping): Finding Location of "<<destination<<endl;
    }
    double getEfficiencyFactor() const override { return 0.95; }
};
```

```
342 class RescueDrone: public Drone{
343 public:
344     RescueDrone(string id,string model,double battery,double range,
345                         double payload,double speed):Drone(id,model,battery,range,payload,speed){}
346     string gettypename() const override{ return "Rescue";}
347     void performMission(const string& destination) override{
348         cout<<getId()<<"(Rescue): Help Arriving "<<destination<<endl;
349     }
350     double getEfficiencyFactor() const override { return 1.25; }
351 };
352 // Builds the right drone class from its type name (used when loading files)
353 unique_ptr<Drone> createDrone(const string& type,const string& id,const string& model,
354                               double battery,double range,double payload,double speed){
355     if(type=="Delivery")
356         return make_unique<DeliveryDrone>(id,model,battery,range,payload,speed);
357     if(type=="Surveillance")
358         return make_unique<SurveillanceDrone>(id,model,battery,range,payload,speed);
359     if(type=="Mapping")
360         return make_unique<MappingDrone>(id,model,battery,range,payload,speed);
361     if(type=="Rescue")
362         return make_unique<RescueDrone>(id,model,battery,range,payload,speed);
363     throw invalid_argument("Unknown drone type: "+type);
364 }
365 /*a small struct above FleetManager to carry the outcome back and why each drone was rejected*/
366 struct AssignmentResult {
367     bool assigned = false;
368     string droneId;
369     vector<string> reasons;    // why each drone was rejected
370 };
371 class MaintenanceRecord {
372     string droneId;
373     string date;
374     string description;
375 public:
376     // new record: the date is stamped automatically
377     MaintenanceRecord(string droneId, string description)
378         : droneId(droneId), date(currentTime()), description(description) {}
379     // used when loading a saved record, so the original date is kept
380     MaintenanceRecord(string droneId, string description, string date)
381         : droneId(droneId), date(date), description(description) {}
382     string getDroneId() const { return droneId; }
383     string getDate() const { return date; }
384     string getDescription() const { return description; }
385 };
386 struct CompletionResult {
387     string droneId;
388     double batteryUsed;
389     double batteryLeft;
390 };
391 struct LoadSummary {
392     int drones = 0, missions = 0, records = 0, skipped = 0;
393 };
394 class FleetManager{
395     map<string,unique_ptr<Drone>> drones;
396     map<string, Mission> missions;       // key = mission ID
397     int nextSequence = 1;
398     vector<MaintenanceRecord> maintenanceLog;
399 public:
400     bool hasDrone(const string& id) const { return drones.count(id)>0;}
401     size_t count() const{ return drones.size();}
402     void addDrone(unique_ptr<Drone> drone){
403         string id=drone->getId();
404         if(hasDrone(id))
405             throw invalid_argument("Drone ID "+ id +" already exists");
406         drones[id]=move(drone);
407     }
408     const Drone* findDrone(const string& id) const{
409         auto it=drones.find(id);
410         if(it==drones.end()) return nullptr;
411         return it->second.get();
412     }
413     void removeDrone(const string& id){
414         auto it=drones.find(id);
415         if(it==drones.end())
416             throw invalid_argument("No drone found with ID "+id);
417         if(it->second->getStatus()==DroneStatus::ON_MISSION)
418             throw runtime_error("Drone "+id+" is on a mission and cannot be removed");
419         drones.erase(it);
420     }
421     void displayAll() const{
422         if(drones.empty()){
423             cout<<"No drones registered yet."<<endl;
424             return;
425         }
426         printDroneHeader();
427         for(const auto&entry : drones)
428             entry.second->display();
429     }
430     //-----missions-------
431     bool hasMission(const string& id)const {return missions.count(id)>0;}
432     size_t missionCount() const{return missions.size();}
433     void addMission(const string& id,MissionType type,const string& destination,double distance
434                     ,double payload,Priority priority){
435         if(hasMission(id))
436             throw invalid_argument("Mission ID "+id+" already exists");
437         missions.emplace(id,Mission(id,type,destination,distance,payload,
438                                     priority,nextSequence));
439         nextSequence++;      // only reached if the Mission was created successfully
440     }
441     void displayAllMissions() const{
442         if(missions.empty()){
443             cout<<"No missions created yet."<<endl;
```

```
            return;
        }
        printMissionHeader();
        for(const auto& entry : missions)
            entry.second.display();
    }
    size_t displayPendingMissions() const {
        size_t shown = 0;
        for (const auto& entry : missions) {
            if (entry.second.getStatus() != MissionStatus::PENDING) continue;
            if (shown == 0) printMissionHeader();
            entry.second.display();
            shown++;
        }
        return shown;
    }
    // priority_queue keeps the most urgent PENDING mission on top
    string highestPriorityPendingId() const {
        struct Entry { Priority priority; int sequence; string id; };
        struct Order {
            bool operator()(const Entry& a, const Entry& b) const {
                if (a.priority != b.priority) return a.priority < b.priority;  // higher priority first
                return a.sequence > b.sequence;                                // then older mission first
            }
        };
        priority_queue<Entry, vector<Entry>, Order> queue;
        for (const auto& entry : missions)
            if (entry.second.getStatus() == MissionStatus::PENDING)
                queue.push({entry.second.getPriority(), entry.second.getSequence(), entry.first});
        return queue.empty() ? "" : queue.top().id;
    }
    AssignmentResult assignMission(const string& missionId) {
        auto mit = missions.find(missionId);
        if (mit == missions.end())
            throw invalid_argument("No mission found with ID " + missionId);
        Mission& mission = mit->second;
        if (mission.getStatus() != MissionStatus::PENDING)
            throw runtime_error("Mission " + missionId + " is already " +
                                missionStatusToString(mission.getStatus()));
        AssignmentResult result;
        Drone* best = nullptr;
        for (auto& entry : drones) {
            Drone* candidate = entry.second.get();
            string reason = candidate->checkSuitability(mission);
            if (!reason.empty()) {
                result.reasons.push_back(candidate->getId() + ": " + reason);
                continue;
            }
            if (best == nullptr) { best = candidate; continue; }
            // prefer a drone whose type matches the mission, then the higher battery
            string wanted = missionTypeToString(mission.getType());
            bool candMatch = candidate->gettypename() == wanted;
            bool bestMatch = best->gettypename() == wanted;
            if (candMatch != bestMatch) { if (candMatch) best = candidate; }
            else if (candidate->getBattery() > best->getBattery()) best = candidate;
        }
        if (best == nullptr) return result;      // mission stays PENDING
        best->setStatus(DroneStatus::ON_MISSION);
        mission.assignTo(best->getId());
        result.assigned = true;
        result.droneId = best->getId();
        return result;
    }
    // ===== Stage 7: complete, charge, maintenance =====
    size_t displayAssignedMissions() const {
        size_t shown = 0;
        for (const auto& entry : missions) {
            if (entry.second.getStatus() != MissionStatus::ASSIGNED) continue;
            if (shown == 0) printMissionHeader();
            entry.second.display();
            shown++;
        }
        return shown;
    }
    CompletionResult completeMission(const string& missionId) {
        auto mit = missions.find(missionId);
        if (mit == missions.end())
            throw invalid_argument("No mission found with ID " + missionId);
        Mission& mission = mit->second;
        if (mission.getStatus() != MissionStatus::ASSIGNED)
            throw runtime_error("Mission " + missionId + " is " +
                                missionStatusToString(mission.getStatus()) +
                                "; only ASSIGNED missions can be completed");
        Drone& drone = requireDrone(mission.getDroneId());
        double before = drone.getBattery();
        double used = drone.estimateConsumption(mission);
        drone.performMission(mission.getDestination());     // type-specific message
        drone.useBattery(used);                              // never goes below 0
        mission.markCompleted();
        drone.setStatus(DroneStatus::AVAILABLE);
        return {drone.getId(), before - drone.getBattery(), drone.getBattery()};
    }
    double chargeDrone(const string& id, double amount) {
        Drone& drone = requireDrone(id);
        if (amount <= 0 || amount > 100)
            throw invalid_argument("Charge amount must be above 0 and at most 100");
        if (drone.getStatus() == DroneStatus::ON_MISSION)
            throw runtime_error("Drone " + id + " is on a mission and cannot be charged");
        if (drone.getStatus() == DroneStatus::MAINTENANCE)
            throw runtime_error("Drone " + id + " is under maintenance and cannot be charged");
        if (drone.getBattery() >= 100)
            throw runtime_error("Drone " + id + " is already fully charged");
```

```
571         drone.charge(amount);                               // capped at 100 inside Drone
572         return drone.getBattery();
573     }
575     bool isUnderMaintenance(const string& id) const {
576         const Drone* d = findDrone(id);
577         return d != nullptr && d->getStatus() == DroneStatus::MAINTENANCE;
578     }
580     void startMaintenance(const string& id, const string& description) {
581         Drone& drone = requireDrone(id);
582         if (drone.getStatus() == DroneStatus::ON_MISSION)
583             throw runtime_error("Drone " + id + " is on a mission; complete it first");
584         if (drone.getStatus() == DroneStatus::MAINTENANCE)
585             throw runtime_error("Drone " + id + " is already under maintenance");
586         drone.setStatus(DroneStatus::MAINTENANCE);
587         maintenanceLog.push_back(MaintenanceRecord(id, description));
588     }
590     void finishMaintenance(const string& id) {
591         Drone& drone = requireDrone(id);
592         if (drone.getStatus() != DroneStatus::MAINTENANCE)
593             throw runtime_error("Drone " + id + " is not under maintenance");
594         drone.setStatus(DroneStatus::AVAILABLE);
595     }
597     void displayMaintenanceHistory(const string& id) const {
598         bool headerShown = false;
599         for (const auto& r : maintenanceLog) {
600             if (r.getDroneId() != id) continue;
601             if (!headerShown) {
602                 cout << "\nMaintenance history of " << id << ":" << endl;
603                 headerShown = true;
604             }
605             cout << "  " << r.getDate() << "  " << r.getDescription() << endl;
606         }
607     }
609     // ===== Stage 8: statistics and files =====
610     void displayStatistics() const {
611         if (drones.empty() && missions.empty()) {
612             cout << "The fleet is empty. Register drones and create missions first." << endl;
613             return;
614         }
616         cout << "DRONES" << endl;
617         cout << "  Total registered      : " << drones.size() << endl;
618         const char* types[] = {"Delivery", "Surveillance", "Mapping", "Rescue"};
619         for (const char* t : types) {
620             int n = 0;
621             for (const auto& e : drones)
622                 if (e.second->gettypename() == t) n++;
623             cout << "    " << left << setw(18) << t << ": " << n << endl;
624         }
625         int available = 0, onMission = 0, maintenance = 0;
626         double totalBattery = 0;
627         for (const auto& e : drones) {
628             DroneStatus st = e.second->getStatus();
629             if (st == DroneStatus::AVAILABLE) available++;
630             else if (st == DroneStatus::ON_MISSION) onMission++;
631             else maintenance++;
632             totalBattery += e.second->getBattery();
633         }
634         cout << "  AVAILABLE             : " << available << endl;
635         cout << "  ON_MISSION            : " << onMission << endl;
636         cout << "  MAINTENANCE           : " << maintenance << endl;
637         if (!drones.empty())
638             cout << "  Average battery       : " << fmt(totalBattery / drones.size()) << " %" << endl;
640         int pending = 0, assigned = 0, completed = 0;
641         double distanceFlown = 0;
642         map<string, int> completedBy;                 // drone ID -> missions completed
643         for (const auto& e : missions) {
644             const Mission& m = e.second;
645             if (m.getStatus() == MissionStatus::PENDING) pending++;
646             else if (m.getStatus() == MissionStatus::ASSIGNED) assigned++;
647             else {
648                 completed++;
649                 distanceFlown += m.getDistance();
650                 completedBy[m.getDroneId()]++;
651             }
652         }
653         cout << "\nMISSIONS" << endl;
654         cout << "  Total created         : " << missions.size() << endl;
655         cout << "  PENDING               : " << pending << endl;
656         cout << "  ASSIGNED              : " << assigned << endl;
657         cout << "  COMPLETED             : " << completed << endl;
658         cout << "  Distance flown        : " << fmt(distanceFlown) << " km (simulated)" << endl;
659         string busiest;
660         int most = 0;
661         for (const auto& e : completedBy)
662             if (e.second > most) { most = e.second; busiest = e.first; }
663         if (most > 0)
664             cout << "  Most active drone     : " << busiest << " (" << most << " missions)" << endl;
666         cout << "\nMAINTENANCE" << endl;
667         cout << "  Records logged        : " << maintenanceLog.size() << endl;
668     }
670     void save() const {
671         ofstream dronesFile("drones.txt");
672         if (!dronesFile) throw runtime_error("Could not write drones.txt");
673         dronesFile << setprecision(10);
674         for (const auto& e : drones) {
675             const Drone& d = *e.second;
676             dronesFile << d.gettypename() << '|' << d.getId() << '|' << d.getModel() << '|'
677                        << d.getBattery() << '|' << d.Range() << '|' << d.Payload() << '|'
678                        << d.Speed() << '|' << static_cast<int>(d.getStatus()) << '\n';
679         }
681         ofstream missionsFile("missions.txt");
682         if (!missionsFile) throw runtime_error("Could not write missions.txt");
```

```
        missionsFile << setprecision(10);
        for (const auto& e : missions) {
            const Mission& m = e.second;
            missionsFile << m.getId() << '|' << static_cast<int>(m.getType()) << '|'
                         << m.getDestination() << '|' << m.getDistance() << '|'
                         << m.getPayload() << '|' << static_cast<int>(m.getPriority()) << '|'
                         << static_cast<int>(m.getStatus()) << '|' << m.getDroneId() << '|'
                         << m.getSequence() << '\n';
        }
        ofstream recordsFile("maintenance.txt");
        if (!recordsFile) throw runtime_error("Could not write maintenance.txt");
        for (const auto& r : maintenanceLog)
            recordsFile << r.getDroneId() << '|' << r.getDate() << '|' << r.getDescription() << '\n';
    }
    LoadSummary load() {
        LoadSummary summary;
        string line;
        ifstream dronesFile("drones.txt");           // a missing file simply reads nothing
        while (getline(dronesFile, line)) {
            if (line.empty()) continue;
            try { loadDroneLine(line); summary.drones++; }
            catch (const exception&) { summary.skipped++; }
        }
        ifstream missionsFile("missions.txt");
        while (getline(missionsFile, line)) {
            if (line.empty()) continue;
            try { loadMissionLine(line); summary.missions++; }
            catch (const exception&) { summary.skipped++; }
        }
        ifstream recordsFile("maintenance.txt");
        while (getline(recordsFile, line)) {
            if (line.empty()) continue;
            vector<string> f = split(line, '|');
            if (f.size() == 3) {
                maintenanceLog.push_back(MaintenanceRecord(f[0], f[2], f[1]));
                summary.records++;
            } else {
                summary.skipped++;
            }
        }
        reconcile();
        return summary;
    }
private:
    Drone& requireDrone(const string& id) {
        auto it = drones.find(id);
        if (it == drones.end())
            throw invalid_argument("No drone found with ID " + id);
        return *it->second;
    }
    void loadDroneLine(const string& line) {
        vector<string> f = split(line, '|');
        if (f.size() != 8) throw invalid_argument("bad drone line");
        int status = parseInt(f[7]);
        if (status < 0 || status > 2) throw invalid_argument("bad status");
        unique_ptr<Drone> d = createDrone(f[0], f[1], f[2], parseNumber(f[3]),
                                          parseNumber(f[4]), parseNumber(f[5]), parseNumber(f[6]));
        d->setStatus(static_cast<DroneStatus>(status));
        addDrone(move(d));                           // throws on a duplicate ID
    }
    void loadMissionLine(const string& line) {
        vector<string> f = split(line, '|');
        if (f.size() != 9) throw invalid_argument("bad mission line");
        int type = parseInt(f[1]), priority = parseInt(f[5]);
        int status = parseInt(f[6]), sequence = parseInt(f[8]);
        if (type < 1 || type > 4 || priority < 1 || priority > 3 || status < 0 || status > 2)
            throw invalid_argument("bad mission value");
        if (hasMission(f[0])) throw invalid_argument("duplicate mission");
        Mission m(f[0], static_cast<MissionType>(type), f[2], parseNumber(f[3]),
                  parseNumber(f[4]), static_cast<Priority>(priority), sequence);
        if (status != 0) m.assignTo(f[7]);           // ASSIGNED or COMPLETED
        if (status == 2) m.markCompleted();
        missions.emplace(m.getId(), m);
        if (sequence >= nextSequence) nextSequence = sequence + 1;
    }
    // Repairs links between drones and missions after loading (e.g. a hand-edited file)
    void reconcile() {
        for (auto& e : missions) {
            Mission& m = e.second;
            if (m.getStatus() != MissionStatus::ASSIGNED) continue;
            auto it = drones.find(m.getDroneId());
            if (it == drones.end())
                m.resetToPending();                          // its drone no longer exists
            else
                it->second->setStatus(DroneStatus::ON_MISSION);
        }
        for (auto& e : drones) {
            if (e.second->getStatus() != DroneStatus::ON_MISSION) continue;
            bool hasMissionAssigned = false;
            for (const auto& me : missions)
                if (me.second.getStatus() == MissionStatus::ASSIGNED &&
                    me.second.getDroneId() == e.first)
                    hasMissionAssigned = true;
            if (!hasMissionAssigned) e.second->setStatus(DroneStatus::AVAILABLE);
        }
    }
};
// ===================== menu actions =====================
void createMission(FleetManager& fleet){
    string id=toUpper(readText("Mission ID         : "));
    if(fleet.hasMission(id))
        throw invalid_argument("Mission ID "+id+" already exists");
```

```
    cout<<"  1. Delivery  2. Surveillance  3. Mapping  4. Rescue"<<endl;
    int type=readInt("Mission type (1-4)  : ",1,4);
    string destination=readText("Destination        : ");
    double distance = readNumber("Distance (km)      : ");
    double payload  = readNumber("Payload (kg)       : ");
    cout<<"  1. Low  2. Medium  3. High"<<endl;
    int priority=readInt("Priority (1-3)     : ",1,3);
    fleet.addMission(id,static_cast<MissionType>(type),destination,distance,payload,static_cast<Priority>(priority));
    cout<<"Mission "<<id<<" created with status PENDING."<<endl;
}
//registerDrone(FleetManager& fleet)
void registerDrone(FleetManager& fleet){
    string id=toUpper(readText("Drone ID           : "));
    if(fleet.hasDrone(id))
        throw invalid_argument("Drone ID "+ id+" already exists");
    string model=readText("Name / Model       : ");
    cout<<"  1. Delivery  2. Surveillance  3. Mapping  4. Rescue"<<endl;
    int type=readInt("Type (1-4)         : ",1,4);
    double battery=readNumber("Battery (%)        : ");
    double range =readNumber("Max range (km)     : ");
    double payload = readNumber("Max payload (kg)    : ");
    double speed= readNumber("Max speed (km/h)    : ");
    unique_ptr<Drone> drone;
    switch(type){
        case 1: drone = make_unique<DeliveryDrone>(id,model,battery,range,payload,speed);break;
        case 2: drone = make_unique<SurveillanceDrone>(id,model,battery,range,payload,speed); break;
        case 3: drone=make_unique<MappingDrone>(id,model,battery,range,payload,speed); break;
        case 4: drone=make_unique<RescueDrone>(id,model,battery,range,payload,speed);break;
    }
    fleet.addDrone(move(drone));
    cout<<"Drone "<<id<<" registered successfully."<<endl;
}
void searchDrone(const FleetManager& fleet) {
    string id = toUpper(readText("Enter Drone ID to search: "));
    const Drone* d = fleet.findDrone(id);
    if (d == nullptr) {
        cout << "No drone found with ID " << id << "." << endl;
        return;
    }
    printDroneHeader();
    d->display();
}
void removeDrone(FleetManager& fleet) {
    if (fleet.count() == 0) {
        cout << "There are no drones to remove." << endl;
        return;
    }
    fleet.displayAll();
    string id = toUpper(readText("\nEnter Drone ID to remove: "));
    string answer = toUpper(readText("Remove drone " + id + "? (y/n): "));
    if (answer != "Y") {
        cout << "Removal cancelled." << endl;
        return;
    }
    fleet.removeDrone(id);
    cout << "Drone " << id << " removed." << endl;
}
void assignMission(FleetManager& fleet) {
    if (fleet.displayPendingMissions() == 0) {
        cout << "There are no PENDING missions to assign." << endl;
        return;
    }
    string id = toUpper(readOptionalText("\nMission ID (Enter = highest priority): "));
    if (id.empty()) {
        id = fleet.highestPriorityPendingId();
        cout << "Selected highest-priority mission: " << id << endl;
    }
    AssignmentResult result = fleet.assignMission(id);
    if (result.assigned) {
        cout << "Mission " << id << " is now ASSIGNED to drone " << result.droneId
             << " (drone status: ON_MISSION)." << endl;
    } else {
        cout << "No suitable drone found. Mission " << id << " remains PENDING." << endl;
        for (const string& reason : result.reasons)
            cout << "   - " << reason << endl;
        if (result.reasons.empty())
            cout << "   - no drones are registered" << endl;
    }
}
void completeMission(FleetManager& fleet) {
    if (fleet.displayAssignedMissions() == 0) {
        cout << "There are no ASSIGNED missions to complete." << endl;
        return;
    }
    string id = toUpper(readText("\nEnter Mission ID to complete: "));
    CompletionResult r = fleet.completeMission(id);
    cout << "Mission " << id << " marked COMPLETED." << endl;
    cout << "Drone " << r.droneId << " used " << r.batteryUsed << "% battery; "
         << r.batteryLeft << "% remaining. Status: AVAILABLE." << endl;
    cout << "(Battery use is a simplified simulation, not real flight data.)" << endl;
    if (r.batteryLeft < 30)
        cout << "Warning: battery is low. Consider charging drone " << r.droneId << "." << endl;
}
void chargeDrone(FleetManager& fleet) {
    if (fleet.count() == 0) {
        cout << "There are no drones to charge." << endl;
        return;
    }
    fleet.displayAll();
    string id = toUpper(readText("\nEnter Drone ID to charge: "));
    double amount = readNumber("Charge to add (%)       : ");
    double now = fleet.chargeDrone(id, amount);
```

```
    cout << "Drone " << id << " charged. Battery is now " << now << "%." << endl;
}
void recordMaintenance(FleetManager& fleet) {
    if (fleet.count() == 0) {
        cout << "There are no drones registered." << endl;
        return;
    }
    fleet.displayAll();
    string id = toUpper(readText("\nEnter Drone ID: "));
    if (!fleet.hasDrone(id))
        throw invalid_argument("No drone found with ID " + id);
    fleet.displayMaintenanceHistory(id);
    if (fleet.isUnderMaintenance(id)) {
        cout << "Drone " << id << " is currently under maintenance." << endl;
        if (readYesNo("Mark maintenance as finished?")) {
            fleet.finishMaintenance(id);
            cout << "Drone " << id << " is AVAILABLE again." << endl;
        }
        return;
    }
    string work = readText("Maintenance details: ");
    fleet.startMaintenance(id, work);
    cout << "Maintenance recorded. Drone " << id
         << " is now under MAINTENANCE and cannot be assigned missions." << endl;
}
// ----- Stage 8 menu helpers -----
void loadRecords(FleetManager& fleet) {
    LoadSummary s = fleet.load();
    if (s.drones + s.missions + s.records + s.skipped == 0) {
        cout << "No saved records found. Starting with an empty fleet." << endl;
        return;
    }
    cout << "Loaded " << s.drones << " drone(s), " << s.missions << " mission(s) and "
         << s.records << " maintenance record(s)." << endl;
    if (s.skipped > 0)
        cout << "Warning: " << s.skipped << " unreadable line(s) were skipped." << endl;
}
void saveRecords(const FleetManager& fleet) {
    try {
        fleet.save();
        cout << "Records saved to drones.txt, missions.txt and maintenance.txt." << endl;
    } catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }
}
void showMenu() {
    cout << "----------------------------------------------------------------" << endl;
    cout << "               DRONE FLEET MANAGEMENT SYSTEM               " << endl;
    cout << "----------------------------------------------------------------" << endl;
    cout << "1. Register Drone" << endl;
    cout << "2. View All Drones" << endl;
    cout << "3. Search Drone" << endl;
    cout << "4. Remove Drone" << endl;
    cout << "5. Create Mission" << endl;
    cout << "6. View All Missions" << endl;
    cout << "7. Assign Mission" << endl;
    cout << "8. Complete Mission" << endl;
    cout << "9. Charge Drone" << endl;
    cout << "10. Record Maintenance" << endl;
    cout << "11. Fleet Statistics" << endl;
    cout << "12. Save Data" << endl;
    cout << "0. Exit" << endl;
    cout << endl;
}
int main(){
    FleetManager fleet;
    loadRecords(fleet);                     // read saved files at start-up
    int n=-1;
    try{
        do{
            showMenu();
            n=readInt("Enter your choice: ",0,12);
            cout<<endl;
            try{
                switch(n){
                    case 1: registerDrone(fleet);break;
                    case 2: fleet.displayAll();break;
                    case 3: searchDrone(fleet);break;
                    case 4: removeDrone(fleet);break;
                    case 5: createMission(fleet);break;
                    case 6: fleet.displayAllMissions();break;
                    case 7: assignMission(fleet);break;
                    case 8: completeMission(fleet);break;
                    case 9: chargeDrone(fleet);break;
                    case 10: recordMaintenance(fleet);break;
                    case 11: fleet.displayStatistics();break;
                    case 12: saveRecords(fleet);break;
                    case 0: cout<<"Saving records and exiting. Goodbye!"<<endl;break;
                }
            }catch(const exception& e){
                cout<<"Error: "<<e.what()<<endl;
            }
            cout<<endl;
        }while(n!=0);
    }catch(const InputClosed&){
        cout<<"\nInput closed. Saving records before exit."<<endl;
    }
    saveRecords(fleet);                     // always save on the way out
    return 0;
}
```

17. Output

The screens below were produced by running the program in a terminal (the text after each prompt is the user's input). The sessions follow each other: records saved at exit were loaded automatically in the next run. Long tables that repeat are shortened as marked.

Output 1: Main menu (first run, no saved records)

```
No saved records found. Starting with an empty fleet.
----------------------------------------------------------------
               DRONE FLEET MANAGEMENT SYSTEM
----------------------------------------------------------------
1. Register Drone
2. View All Drones
3. Search Drone
4. Remove Drone
5. Create Mission
6. View All Missions
7. Assign Mission
8. Complete Mission
9. Charge Drone
10. Record Maintenance
11. Fleet Statistics
12. Save Data
0. Exit

Enter your choice: 1
```

Output 2: Registering a drone (D1 shown; S1 and R1 were registered the same way) and viewing the fleet

```
Enter your choice: 1

Drone ID            : d1
Name / Model        : Falcon X
  1. Delivery  2. Surveillance  3. Mapping  4. Rescue
Type (1-4)          : 1
Battery (%)         : 90
Max range (km)      : 20
Max payload (kg)    : 5
Max speed (km/h)    : 60
Drone D1 registered successfully.
Enter your choice: 2

ID      Model         Type          Batt%   Range   Payload Speed   Status
-------------------------------------------------------------------------------
D1      Falcon X      Delivery      90      20      5       60      AVAILABLE
R1      Lifeguard     Rescue        100     30      10      80      AVAILABLE
S1      Eagle Eye     Surveillance  80      15      1       50      AVAILABLE
```

Output 3: Creating a mission (M1 shown; M2 and M3 were created the same way) and viewing all missions

```
Enter your choice: 5

Mission ID          : m1
  1. Delivery  2. Surveillance  3. Mapping  4. Rescue
Mission type (1-4)  : 1
Destination         : Sector 7
Distance (km)       : 8
Payload (kg)        : 2
  1. Low  2. Medium  3. High
Priority (1-3)      : 3
Mission M1 created with status PENDING.
Enter your choice: 6

ID      Type          Destination     Dist    Payload  Priority  Status     Drone
-------------------------------------------------------------------------------
M1      Delivery      Sector 7        8       2        HIGH      PENDING    -
M2      Rescue        Flood Zone      12      4        MEDIUM    PENDING    -
M3      Mapping       Far Ridge       40      0        LOW       PENDING    -
```

Output 4: Assigning missions in priority order, and a mission that no drone can take

```
Enter your choice: 7

ID      Type          Destination     Dist    Payload  Priority  Status     Drone
-------------------------------------------------------------------------------
M1      Delivery      Sector 7        8       2        HIGH      PENDING    -
M2      Rescue        Flood Zone      12      4        MEDIUM    PENDING    -
M3      Mapping       Far Ridge       40      0        LOW       PENDING    -

Mission ID (Enter = highest priority):
Selected highest-priority mission: M1
Mission M1 is now ASSIGNED to drone D1 (drone status: ON_MISSION).

Enter your choice: 7

[mission list displayed here]
Mission ID (Enter = highest priority):
Selected highest-priority mission: M2
Mission M2 is now ASSIGNED to drone R1 (drone status: ON_MISSION).

Enter your choice: 7

[mission list displayed here]

Mission ID (Enter = highest priority):
Selected highest-priority mission: M3
No suitable drone found. Mission M3 remains PENDING.
   - D1: not available (ON_MISSION)
   - R1: not available (ON_MISSION)
   - S1: range too short (max 15.0 km)
```

Output 5: Completing a mission, charging a drone and recording maintenance

```
Enter your choice: 8

ID      Type          Destination     Dist    Payload  Priority  Status     Drone
-------------------------------------------------------------------------------
M1      Delivery      Sector 7        8       2        HIGH      ASSIGNED   D1
M2      Rescue        Flood Zone      12      4        MEDIUM    ASSIGNED   R1

Enter Mission ID to complete: m1
D1(Delivery): delivering package to Sector 7
Mission M1 marked COMPLETED.
Drone D1 used 52.8% battery; 37.2% remaining. Status: AVAILABLE.
(Battery use is a simplified simulation, not real flight data.)

Enter your choice: 9

[drone list displayed here]
Enter Drone ID to charge: d1
Charge to add (%)       : 40
Drone D1 charged. Battery is now 77.2%.

Enter your choice: 10

[drone list displayed here]

Enter Drone ID: s1
Maintenance details: Camera check
Maintenance recorded. Drone S1 is now under MAINTENANCE and cannot be assigned missions.
```

Output 6: Fleet statistics

```
Enter your choice: 11

DRONES
  Total registered      : 3
    Delivery          : 1
    Surveillance      : 1
    Mapping           : 0
    Rescue            : 1
  AVAILABLE             : 1
  ON_MISSION            : 1
  MAINTENANCE           : 1
  Average battery       : 85.7 %

MISSIONS
  Total created         : 3
  PENDING               : 1
  ASSIGNED              : 1
  COMPLETED             : 1
  Distance flown        : 8.0 km (simulated)
  Most active drone     : D1 (1 missions)

MAINTENANCE
  Records logged        : 1
```

Output 7: Next run - records loaded automatically from the files

```
Loaded 3 drone(s), 3 mission(s) and 1 maintenance record(s).
Enter your choice: 2

ID      Model         Type          Batt%   Range   Payload Speed   Status
-------------------------------------------------------------------------------
D1      Falcon X      Delivery      77.2    20      5       60      AVAILABLE
R1      Lifeguard     Rescue        100     30      10      80      ON_MISSION
S1      Eagle Eye     Surveillance  80      15      1       50      MAINTENANCE
```

Output 8: Content of the saved files

```
[drones.txt]
Delivery|D1|Falcon X|77.2|20|5|60|0
Rescue|R1|Lifeguard|100|30|10|80|1
Surveillance|S1|Eagle Eye|80|15|1|50|0
[missions.txt]
M1|1|Sector 7|8|2|3|2|D1|1
M2|4|Flood Zone|12|4|2|1|R1|2
M3|3|Far Ridge|40|0|1|0||3
[maintenance.txt]
S1|2026-10-28 00:59|Camera check
```

Output 9: Error handling (invalid menu input, duplicate ID, invalid battery)

```
Enter your choice: abc
Please enter a whole number from 0 to 12.

Enter your choice: 1

Drone ID            : D1
Error: Drone ID D1 already exists

Enter your choice: 1

Drone ID            : x9
Name / Model        : Test
  1. Delivery  2. Surveillance  3. Mapping  4. Rescue
Type (1-4)          : 1
Battery (%)         : 150
Max range (km)      : 20
Max payload (kg)    : 5
Max speed (km/h)    : 60
Error: Battery must be 0-100
```

Output 10: Searching for a drone (found and not found)

```
Enter your choice: 3

Enter Drone ID to search: r1
ID      Model         Type          Batt%   Range   Payload Speed   Status
-------------------------------------------------------------------------------
R1      Lifeguard     Rescue        100     30      10      80      ON_MISSION

Enter your choice: 3

Enter Drone ID to search: zz
No drone found with ID ZZ.
```

Output 11: Removing a drone (refused while ON_MISSION; cancelled by the user)

```
Enter your choice: 4

[drone list displayed here]

Enter Drone ID to remove: r1
Remove drone R1? (y/n): y
Error: Drone R1 is on a mission and cannot be removed

Enter your choice: 4

[drone list displayed here]

Enter Drone ID to remove: d1
Remove drone D1? (y/n): n
Removal cancelled.
```

Output 12: Charging refused for a drone in maintenance and a drone on a mission

```
Enter your choice: 9

[drone list displayed here]

Enter Drone ID to charge: s1
Charge to add (%)       : 10
Error: Drone S1 is under maintenance and cannot be charged

Enter your choice: 9

[drone list displayed here]

Enter Drone ID to charge: r1
Charge to add (%)       : 10
Error: Drone R1 is on a mission and cannot be charged
```

Output 13: Assignment when drones are busy, in maintenance or out of range

```
Enter your choice: 7

ID      Type          Destination     Dist    Payload  Priority  Status     Drone
-------------------------------------------------------------------------------
M3      Mapping       Far Ridge       40      0        LOW       PENDING    -

Mission ID (Enter = highest priority):
Selected highest-priority mission: M3
No suitable drone found. Mission M3 remains PENDING.
   - D1: range too short (max 20.0 km)
   - R1: not available (ON_MISSION)
   - S1: not available (MAINTENANCE)
```

Output 14: Finishing maintenance makes the drone AVAILABLE again

```
Enter your choice: 10

[drone list displayed here]

Enter Drone ID: s1

Maintenance history of S1:
  2026-10-28 00:59  Camera check
Drone S1 is currently under maintenance.
Mark maintenance as finished? (y/n): y
Drone S1 is AVAILABLE again.
```

18. Testing and Validation

Each stage was compiled with warnings enabled and tested with scripted input sessions that included both valid and invalid input. The final program was also built with AddressSanitizer and UndefinedBehaviorSanitizer, which reported no memory or undefined-behaviour problems.

Test case	Expected behaviour	Result
Letters or out-of-range number in the menu	Message shown, menu asks again	Passed
Duplicate drone or mission ID (d1 and D1)	Error message, nothing added	Passed
Battery 150, negative payload, zero distance, nan, inf	Rejected by input helpers or constructors	Passed
Assign with no missions / no drones	Friendly message, no crash	Passed
Mission too far, too heavy or battery too low	Stays PENDING; reason listed for every drone	Passed
Assign the same mission twice	Error: already ASSIGNED	Passed
Remove, charge or maintain a drone that is ON_MISSION	Refused with an error message	Passed

Test case	Expected behaviour	Result
Charge a drone under maintenance or at 100%	Refused; charging never exceeds 100%	Passed
Drone under maintenance offered a mission	Rejected: not available (MAINTENANCE)	Passed
Complete a mission that is not ASSIGNED / wrong ID	Error message	Passed
Save, close and restart	All drones, missions and records restored	Passed
Corrupted or hand-edited data files	Bad lines skipped and counted; program continues	Passed
Mission assigned to a drone missing from the file	Mission returns to PENDING (reconcile)	Passed
Input stream closed in the middle of a prompt	Program saves and exits without crashing	Passed

19. Limitations and Future Scope

- **Limitations:** single-user and console-only; simplified battery model; text files instead of a database; no weather, flight paths or real-time conditions; all data is simulated.
- **Future scope:** a database for larger fleets, a graphical interface, more detailed energy models, mission scheduling by time, user login with roles, and integration with real telemetry (outside the scope of this academic version).

20. Challenges Faced and Learning Outcomes

Challenge	How it was solved
Mixing cin >> and getline skipped questions because a newline stayed in the buffer	All input is read with getline and converted by helper functions that re-ask until valid
Letters typed instead of numbers could make the menu loop forever; nan and inf passed range checks	readInt/readNumber validate the whole text and reject non-finite numbers
Storing different drone types in one container	An abstract Drone base class and a map of unique_ptr<Drone> with virtual functions
Keeping drone and mission statuses consistent	Both are changed together inside FleetManager; reconcile() repairs saved data
Hand-edited or damaged data files	Every line is validated with try/catch; bad lines are skipped and counted

Learning outcomes

- Designing a program as cooperating classes with clear responsibilities.
- Using inheritance, abstract classes and virtual functions to obtain polymorphic behaviour.
- Applying STL containers, smart pointers, exceptions and file streams in one project.
- Testing in small stages, including invalid input, and reading compiler warnings.

21. Conclusion

The project delivers a working, menu-driven C++ console application that manages a simulated drone fleet, assigns suitable drones to missions through clear rules, updates battery and status information, supports charging and maintenance, reports statistics and saves its records. It demonstrates how encapsulation, inheritance, abstraction, polymorphism, constructors, exception handling, STL containers and file handling can be combined to organise a moderately sized program into reusable, maintainable classes. Building it in stages also showed the value of validating input early, keeping rules in one place and testing after every change.